

UART Transmitter

MIT Fall 2025

The questions below are due on Wednesday September 24, 2025; 11:59:00 PM.

UART Protocol

Last week, we saw the SPI protocol in use, and now it's time to use another serial data protocol: *Universal Asynchronous Receiver/Transmitter*, or UART.

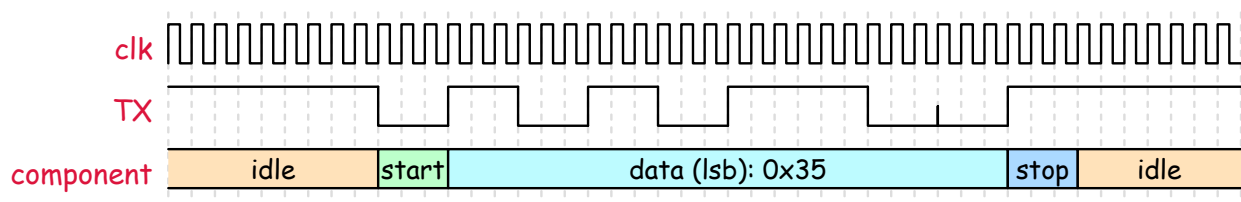


Figure 1 - A serial UART transmission of the byte 0x35. Here, there are three clock periods per Baud period, but usually that value is much larger.

This page will have you build a transmitter for UART, which only needs one wire to transmit data as opposed to the three wires of SPI (data, clock, and chip select)! By moving to fewer wires (a benefit), however, there are sacrifices that need to be made in order for it to work reliably.

Here's a handful of highlights of the protocol that'll be important for implementing it:

- **Asynchronous**

The UART protocol has only two wires, one for transmitting data and one for receiving data. What that means is there's **no clock** to rely on. Instead, both sides of the bus agree on a *BAUD* rate, which defines the number of bits that are transmitted on a wire per second. BAUD rates need to be substantially smaller than the operating clock of the devices on either side of the bus, so that each device can reliably keep time as they send and receive data.

- **Start and Stop Bits**

In order to distinguish when there's silence as opposed to a byte being sent on a serial wire, UART includes a "start" bit (0) and a "stop" bit (1) before and after each byte. The wire is **held high** (logical 1) when not in use to send data, and then the "start" bit (0) indicates that data is about to be sent. The "stop" bit (1) should be sent after the byte is transmitted.

- **Little Endian**

When the bits within a byte are being sent, UART decided (somewhat arbitrarily, but largely now for backwards compatibility reasons/convention) that the first bit sent should be the *least significant* bit of the byte (i.e. byte[0], i.e. the "1's" place of the byte), and then the following bits will each be from a more significant place. This is different from what we did in SPI¹.

Show Queue Controls

Check Yourself: UART

In terms of the input clock speed clk_i and BAUD rate $baud$, how many clock cycles should a single UART bit last for? Use `clk_i` to represent the input clock frequency, and `baud` to represent the BAUD rate.

When you write modules for UART, it may be helpful to define a local parameter `localparam BAUD_BIT_PERIOD` to be equal to this expression.

 100.00%

You have 4 submissions remaining.

Your entry was parsed as:

$$\frac{clk_i}{baud}$$

With a clock speed of 100MHz and a BAUD rate of 4800, how many clock cycles should it take to send one full byte of data? Include the time to send start and stop bits.

 100.00%

You have 4 submissions remaining.

The letter "b" is encoded in ASCII as the byte 0x62. What sequence of bits, **including the start and stop bit**, would transmit the letter "b" over UART?

 100.00%

You have 4 submissions remaining.

Building Your Own Transmitter

Try your hand at making a UART transmitter module based on what we discussed above, that can take in data bytes and send them serially over a UART wire.

Parameters

- parameter `INPUT_CLOCK_FREQ`: frequency, in Hz, of `clk`
- parameter `BAUD_RATE`: bitrate for UART transmission, in bits/second

Inputs and Outputs

- input wire `clk`: operational clock
- input wire `rst`: signal to reset the module (active high)
- input wire `[7:0] din`: (short for "data in") carries the byte that should be transmitted over UART; it should only be read when a transmission is started!
- input wire `trigger`: indicates that a byte transmission should be started, and that `din` contains valid data for the next transmission. **Should be ignored if your module is still busy sending a message!! Don't let a new trigger interrupt an ongoing message.**
- output logic `busy`: should be held high whenever the module is still transmitting a byte and can't accept new data input. We didn't do this in our SPI module, but this is a means of allowing "back-pressure", which is a way of telling the system driving the module that new transmits will not be accepted right now (and to, if needed, hold up its operation). *This should be busy even when sending start and stop bits!*
- output logic `dout`: (short for "data out") serial output signal, as described

Testing

We're into our third week together. It is time that you start to generate your own test code from scratch. Feel free to bring over some of your SPI tester code from last week for at least a boilerplate starter (but of course make sure to update/modify all appropriate fields, variables, and function names. *use the cocotb documentation, and the exercise from week 01, as reference when writing your tests!* A few things to keep in mind while testing:

- Utilize a shift buffer for managing your data. Avoid using an indexing variable. **If the staff finds you using an array with an indexing variable during Checkoff 1, they'll make you rewrite the module before giving you the checkoff.** Refer to [lecture 3](#) and [lecture 4](#) where this is discussed.
- When a device is not transmitting, the data line must be kept high. This includes when being reset. Be very careful to avoid false 0's. These can trigger the listener to think a message has started transmitting and cause bad packets to be received. This can impact not only the next byte sent, but many bytes into the future depending on the contents of the data.
- The stop and start bits are extremely important. They must be full-length (as long as any data bit). Do not neglect them.
- Things are transmitted lsb-first. lsb-first.
- lsb-first.

module uart_transmit

Submit your tested UART transmitter module here.

Code Skeleton

Download Most Recent Submission

Select File

No file selected

Submit

100.00%

You have 7 submissions remaining.

Simulation Tests Completed

Test Summary

3 of 3 tests passed:

- test single_transmit: PASSED
- test transmit_sequence: PASSED
- test transmit_sequence_gap: PASSED

build log

No logged results for build.

test log

```

-.-ns INFO      gpi
-.-ns INFO      gpi
0.00ns INFO      cocotb
0.00ns INFO      cocotb
0.00ns INFO      cocotb
0.00ns INFO      cocotb.regression
0.00ns INFO      cocotb.regression
0.00ns INFO      cocotb.regression
0.00ns INFO      cocotb.regression
0.00ns INFO      cocotb.regression
0.00ns INFO      cocotb.uart_transmit
75.00ns INFO      cocotb.uart_transmit
80.00ns INFO      cocotb.uart_transmit
300.00ns INFO      cocotb.uart_transmit

..mbed/gpi_embed.cpp:108 in set_progra
../gpi/GpiCommon.cpp:101 in gpi_print_
Running on Icarus Verilog version 12.0
Running tests with cocotb v1.9.0 from /
Seeding Python random module with suppl
Found test test_tx.single_transmit
Found test test_tx.transmit_sequence
Found test test_tx.transmit_sequence_ga
running single_transmit (1/3)
    transmit one byte over UART, with an
clock Clock(100.0 MHz)
Requesting transmission of byte 0x42
Beginning of transmission de
Erroneously pulling trigger

```

```

9580.00ns INFO cocotb.uart_transmit Byte received: 0b01000011 / 0x43
11080.00ns INFO cocotb.regression single_transmit passed
11080.00ns INFO cocotb.regression running transmit_sequence (2/3)
    transmit several bytes in rapid succe
    clock Clock(100.0 MHz)
    Requesting transmission of byte 0xfa
    Beginning of transmission detected
    Byte received: 0b1111010 / 0xfa
    Requesting transmission of byte 0xea
    Beginning of transmission detected
    Byte received: 0b11101010 / 0xea
    Requesting transmission of byte 0x22
    Beginning of transmission detected
    Byte received: 0b00100010 / 0x22
    Requesting transmission of byte 0xff
    Beginning of transmission detected
    Byte received: 0b11111111 / 0xff
    Requesting transmission of byte 0x0
    Beginning of transmission detected
    Byte received: 0b00000000 / 0x0
    Requesting transmission of byte 0x1
    Beginning of transmission detected
    Byte received: 0b00000001 / 0x1
    Requesting transmission of byte 0x7
    Beginning of transmission detected
    Byte received: 0b00000111 / 0x7
    Requesting transmission of byte 0x55
    Beginning of transmission detected
    Byte received: 0b01010101 / 0x55
    transmit_sequence passed
    running transmit_sequence_gap (3/3)
    transmit several bytes with gaps in b
    clock Clock(100.0 MHz)
    Requesting transmission of byte 0xfa
    Beginning of transmission detected
    Byte received: 0b1111010 / 0xfa
    Requesting transmission of byte 0xea
    Beginning of transmission detected
    Byte received: 0b11101010 / 0xea
    Requesting transmission of byte 0x22
    Beginning of transmission detected
    Byte received: 0b00100010 / 0x22
    Requesting transmission of byte 0xff
    Beginning of transmission detected
    Byte received: 0b11111111 / 0xff
    Requesting transmission of byte 0x0
    Beginning of transmission detected
    Byte received: 0b00000000 / 0x0
    Requesting transmission of byte 0x1
    Beginning of transmission detected
    Byte received: 0b00000001 / 0x1
    Requesting transmission of byte 0x7
    Beginning of transmission detected
    Byte received: 0b00000111 / 0x7
    transmit_sequence_gap passed
*****
** TEST STATU
*****
** test_tx.single_transmit PASS
** test_tx.transmit_sequence PASS
** test_tx.transmit_sequence_gap PASS
*****
** TESTS=3 PASS=3 FAIL=0 SKIP=0

```

Did you use a shift buffer? A UART transmitter that uses variable indexing may be able to pass this checker, but will not get you a checkoff! Make sure you are using a shift buffer!

Footnotes

¹SPI in general doesn't actually specify whether data needs to be sent MSB-first or LSB-first. That's left up to the individual chips to decide. Most of the time you'll see MSB-first, but it can vary. ([click to return to text](#))

Home > Assignments > Week 03: UART, etc... > UART Transmitter



Powered by [CAT-SOOP v19.0.6+git.3.26e8b68c](#) "Korat" (LTS) (development snapshot).

CAT-SOOP is [Free/Libre Software](#), available under the terms
of the [GNU Affero General Public License, version 3](#).

[Download Source Code](#)

[Javascript License Information](#)